

BTC Market Intelligence Engine — Data Plane

Subsystem brief v2

Architectural role: Hexagonal infrastructure adapter layer

Purpose

Ingest, preserve and replay BTCUSDT perpetual market data with deterministic provenance.

Hexagonal boundary

The Data Plane is infrastructure. It must implement domain/application ports and must not define market-intelligence semantics.

```

Binance
  ↓
Binance Adapter
  ↓
MarketDataProvider port
  ↓
MIE domain core
```

For historical replay:

```

Raw Parquet
  ↓
HistoricalDataAdapter
  ↓
HistoricalDataProvider port
  ↓
same MIE domain core
```

Initial path

Binance → Rust ingestion → immutable raw Parquet → DuckDB replay/research

PostgreSQL and ClickHouse remain deferred until workload proves they are needed.

Raw data candidates

Explicit Order Flow / Aggression Data Contract

The Data Plane must preserve enough exchange-native information to reconstruct aggressive buy/sell flow and order-book interaction.

Required raw inputs:

- individual trades / executions
- trade price and quantity
- aggressor-side information where available, otherwise deterministic reconstruction
- depth/order-book events
- bid/ask liquidity changes
- event timestamps and ordering

The Data Plane must preserve these raw events rather than storing only OHLCV aggregates. Derived aggressive buy/sell volume, delta, CVD, trade intensity and liquidity-response features belong above the raw storage boundary.

Bookmap is a reference visualization model only; the MIE must remain independent of it.

- trades
- depth/order-book events
- klines/OHLCV
- open interest
- funding
- liquidations

Design rules

1. Raw data is the source of truth.
2. Raw records are immutable.
3. Derived features must be reproducible from raw data.
4. Live and replay must expose compatible domain-level inputs.
5. Infrastructure-specific types must not leak into the domain core.
6. Data version and provenance must be recorded for research experiments.

Port responsibilities

`MarketDataProvider` should provide deterministic domain-level observations.

It should not return Binance SDK objects to the core.

`HistoricalDataProvider` should reproduce the information available at the decision moment and preserve event ordering where relevant.

Why this matters

The same Market State, Regime and Strategy logic must be executable against:

- live Binance data
- historical Parquet
- replay
- forward-test feeds
- synthetic research data

without rewriting the domain logic.